

Proyecto Corte 1

Universidad Autónoma de Occidente

Proyecto Corte 1 · Frameworks y Herramientas para Big Data

Integrantes — Grupo 2

- Nicolás Santiago Cuarán Sotelo
- Sebastian Belalcazar Mosquera
- Bryan Fernando Burbano Carvajal
- Michel Dahiana Burgos Santos
- Juan David Daza Rivera

Validación crítica: 3.475.226 registros · 12/12 checks

NYC Yellow Taxi · enero 2025

MinIO · Apache Iceberg · Nessie · ClickHouse · Azure ADLS · dlt

github.com/SEBASBELMOS/nyc-taxi-lakehouse

Septiembre de 2026

1. Resumen ejecutivo

Se construyó un data stack completo sobre Docker Compose: ingesta de un archivo

Parquet público de NYC Yellow Taxi (enero 2025), aterrizaje en un data lake (MinIO),

conversión a tabla Apache Iceberg catalogada en Nessie mediante el protocolo Iceberg

REST, y distribución final hacia Azure ADLS y ClickHouse. Toda la ingesta y carga se

realiza con `dlt`; la lectura de Parquet se hace con PyArrow en lotes.

Resultado de la validación crítica. La tabla final en ClickHouse contiene exactamente **3.475.226 filas**, verificado de forma independiente en tres capas: RAW (MinIO), tabla Iceberg (vía Nessie) y ClickHouse.

La verificación integral del stack cierra en **12/12 checks**, incluida la confirmación de los archivos cargados en Azure.

2. Entregables del enunciado

Entregable	Ubicación
4 scripts de ingesta y cargue de datos	<code>scripts/01..04_*.py</code> , ejecutados como notebooks en Jupyter (<code>notebooks/01..04_*.ipynb</code> , con sus outputs)
1 archivo <code>docker-compose.yml</code>	Raíz del proyecto. Cuatro servicios: <code>minio</code> , <code>nessie</code> , <code>clickhouse</code> , <code>jupyter</code>
1 archivo con los secrets de <code>dlt</code>	<code>dlt/secrets.toml</code> , incluido en la carpeta de entrega y excluido del repositorio público
Documento con los pantallazos	Este documento, sección 7
Servicios corriendo en Docker	Figura 1
Dos buckets de MinIO con sus Parquet	Figuras 2, 3 y 4
Catálogo de Nessie con el namespace	Figura 5
Consulta con cantidad de registros (3475226)	Figura 6
Consulta con las tablas de metadatos en ClickHouse	Figura 7
Consulta de archivo cargado en Azure	Figura 8

3. Arquitectura

Función	Tecnología	Servicio Docker
Data Lake / Object Storage	MinIO	<code>minio</code>
Catálogo	Nessie (Iceberg REST)	<code>nessie</code>
Data Warehouse	ClickHouse	<code>clickhouse</code>
Entorno interactivo	Jupyter	<code>jupyter</code>
File format	Apache Parquet	—

Función	Tecnología	Servicio Docker
Table format	Apache Iceberg	—
Ingesta / carga	dlt	—
Lectura de Parquet	PyArrow	—



4. Configuración y decisiones técnicas

Manejo de secretos

Las credenciales de Azure, MinIO y ClickHouse viven en `dlt/secrets.toml`, excluido del control de versiones y montado como volumen de solo lectura en el contenedor de Jupyter. Ningún script ni notebook contiene credenciales, y ninguna aparece en los outputs guardados. Los parámetros no secretos van en `.env` (`GROUP_FOLDER=GRUPO_2`).

Nessie como catálogo Iceberg REST

Pylceberg no ofrece un tipo de catálogo `nessie`. Nessie expone el protocolo Iceberg REST en la ruta `/iceberg`, de modo que el cliente se conecta con

`RestCatalog`. El parámetro `prefix` no se utiliza: según la documentación de Nessie no funciona con Pylceberg, y la rama se selecciona por la URI.

Catálogo persistente

El version store de Nessie se configuró en RocksDB sobre un volumen. El valor por defecto (`IN_MEMORY`) pierde el namespace y la tabla al reiniciar el contenedor.

Control de memoria

La fuente ronda los 3,5 millones de filas y nunca se carga completa en memoria: se lee

con `ParquetFile.iter_batches(batch_size=250.000)` entregando lotes de PyArrow. Un `read_table().to_pandas()` equivalente consumiría varios GB dentro del contenedor.

Puertos

ClickHouse publica su protocolo nativo en el puerto 9002 del host porque MinIO ocupa el 9000; dentro de la red Docker ClickHouse sigue escuchando en su 9000, que es el que usa

`dlt`.

Nombres legibles y dependencias

Se fijó `dataset_table_separator = "_"` en `dlt/config.toml`, con lo cual la tabla final es `nyc_taxi_yellow_tripdata_2025_01`. Sin ese ajuste, dlt la nombraría `nyc_taxi__yellow_tripdata_2025_01`, con tres guiones bajos. Todas las dependencias están declaradas en `requirements.txt` e instaladas en la imagen mediante el `Dockerfile`; no hay `pip install` dentro de los notebooks.

5Ejecución de los pipelines

Los cuatro pipelines se ejecutaron como notebooks dentro de Jupyter (`notebooks/01..04`, con sus outputs guardados como evidencia). Las versiones

`.py` equivalentes están en `scripts/` y comparten la misma configuración (`scripts/config.py`).

5.1 Pipeline 01 — HTTP → MinIO RAW

- Fuente: `https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2025-01.parquet`
- Destino: `s3://nyc-taxi-raw/raw/yellow_tripdata/`
- Resultado: un archivo Parquet de 72,9 MB (`1788272855.525187.7eef0a56d0.parquet`)

```
Rows landed in MinIO: 3,475,226
OK - RAW bucket matches the expected row count.
```

5.2 Pipeline 02 — MinIO RAW → Iceberg + Nessie

- Tabla creada: `nyc_taxi.yellow_tripdata_2025_01` (20 columnas)
- Location: `s3://nyc-taxi-iceberg/nyc_taxi/yellow_tripdata_2025_01_25fa2203-...`
- Snapshots: 1 · Data files: 14 · Metadata files: 43
- Carga por lotes de 250.000 filas (`iter_batches` + `append`).
Antes de escribir se eliminan las columnas internas de dlt (`_dlt_load_id` , `_dlt_id`) para que la tabla Iceberg sea copia fiel del origen.

```
[OK] Iceberg row count matches - 3,475,226 (expected 3,475,226)
```

5.3 Pipeline 03 — Iceberg → Azure ADLS

- Origen: los data files se resuelven preguntando al catálogo de Nessie (`table.location()`), sin rutas hardcodeadas.
- Destino: `abfss://clase-4-dlt@fhbd.dfs.core.windows.net/GRUPO_2/nyc_taxi/`
- Resultado: data files Parquet (~73 MB) más la metadata de dlt, confirmados mediante verificación programática con `adlfs` usando las mismas credenciales, sin depender del portal de Azure.

5.4 Pipeline 04 — Iceberg → ClickHouse

- Tabla final: `default.nyc_taxi_yellow_tripdata_2025_01`
- Tablas de metadatos de dlt: `nyc_taxi__dlt_loads`, `nyc_taxi__dlt_pipeline_state`, `nyc_taxi__dlt_version`, `nyc_taxi__dlt_sentinel_table`
- Carga completa en 3,02 s. `write_disposition="replace"` garantiza el conteo exacto en cada re-ejecución.

```
SELECT count(*) FROM default.nyc_taxi_yellow_tripdata_2025_01;
-> 3,475,226
[OK] ClickHouse row count matches - 3,475,226 (expected 3,475,226)
```

6 Verificación integral

El script `scripts/verificar.py` (ejecutado también como `notebooks/verificar.ipynb`) recorre todo el stack en una sola corrida y cierra en 12/12 checks.

```
[OK ] bucket 'nyc-taxi-raw' exists
[OK ] bucket 'nyc-taxi-iceberg' exists
[OK ] RAW bucket has Parquet files - 1 file(s)
[OK ] Iceberg bucket has data files - 14 file(s)
[OK ] Iceberg bucket has metadata files - 43 file(s)
[OK ] namespace 'nyc_taxi' exists
[OK ] table 'nyc_taxi.yellow_tripdata_2025_01' registered
[OK ] Iceberg row count matches - 3,475,226 (expected 3,475,226)
[OK ] table 'nyc_taxi_yellow_tripdata_2025_01' exists
[OK ] dlt metadata tables present
[OK ] ClickHouse row count matches - 3,475,226 (expected 3,475,226)
[OK ] files present in Azure - 5 file(s)
```

7. Evidencias visuales

Las nueve capturas siguientes documentan cada punto solicitado por el enunciado. Los archivos originales están en la carpeta `evidencias/` del proyecto.

```
C:\Users\sebas> docker ps
NAME                  IMAGE                                     STATUS      PORTS
cortel-jupyter        cortel-jupyter:1.0                     Up (healthy)    0.0.0.0:8888->8888/tcp
cortel-nessie         ghcr.io/projectnessie/nessie:0.108.4   Up              0.0.0.0:19120->19120/tcp
cortel-clickhouse     clickhouse/clickhouse-server:25.8       Up (healthy)    0.0.0.0:8123->8123/tcp, 0.0.0.0:9002->9000/tcp
cortel-minio          quay.io/minio/minio:RELEASE.2025-09-07T16-13-09Z Up (healthy)    0.0.0.0:9000-9001->9000-9001/tcp
```

Figura 1

Servicios corriendo en Docker

Los cuatro servicios del stack activos: `minio`, `nessie`, `clickhouse` y `jupyter`. El contenedor `minio-init` aparece finalizado (`Exited 0`) porque su única tarea es crear los buckets. Pantallazo requerido: servicios corriendo en Docker · [evidencias/01_docker_services.png](#)

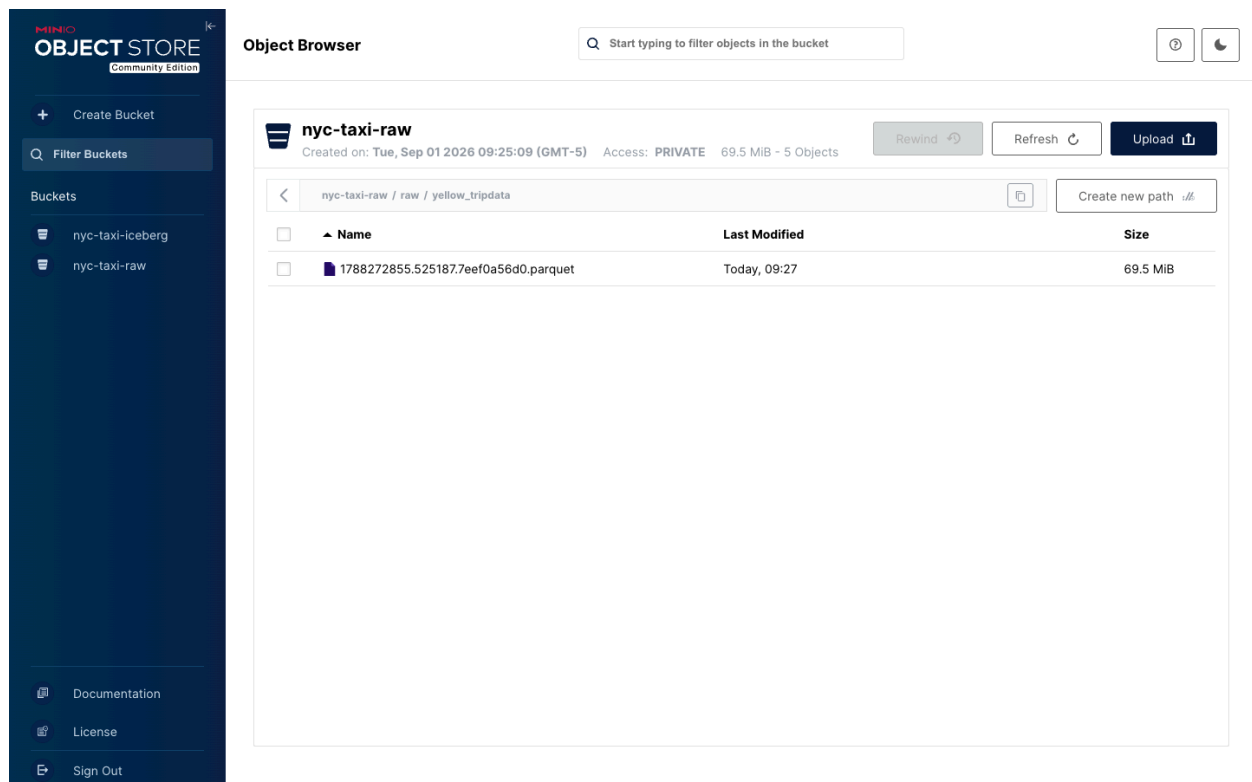


Figura 2

Bucket RAW en MinIO

Bucket `nyc-taxi-raw` con el archivo Parquet aterrizado por el pipeline 01 (72,9 MB, 3.475.226 filas). Pantallazo requerido: buckets de MinIO con sus Parquet (1 de 2) ·

`evidencias/02_minio_raw_bucket.png`

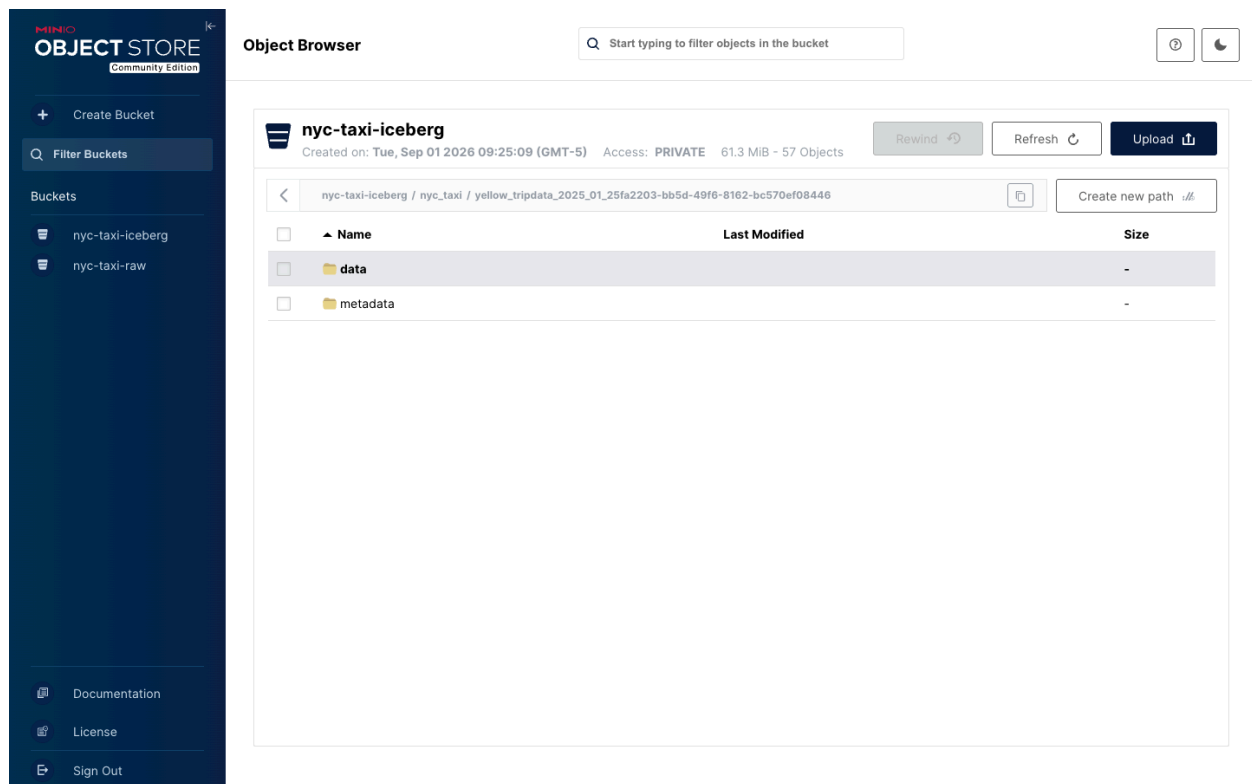


Figura 3

Bucket Iceberg en MinIO

Bucket `nyc-taxi-iceberg` con la estructura de la tabla Apache Iceberg: carpeta `data/` (14 archivos Parquet) y `metadata/` (43 archivos). Pantallazo requerido: buckets de MinIO con sus Parquet (2 de 2) · `evidencias/03_minio_iceberg_bucket.png`

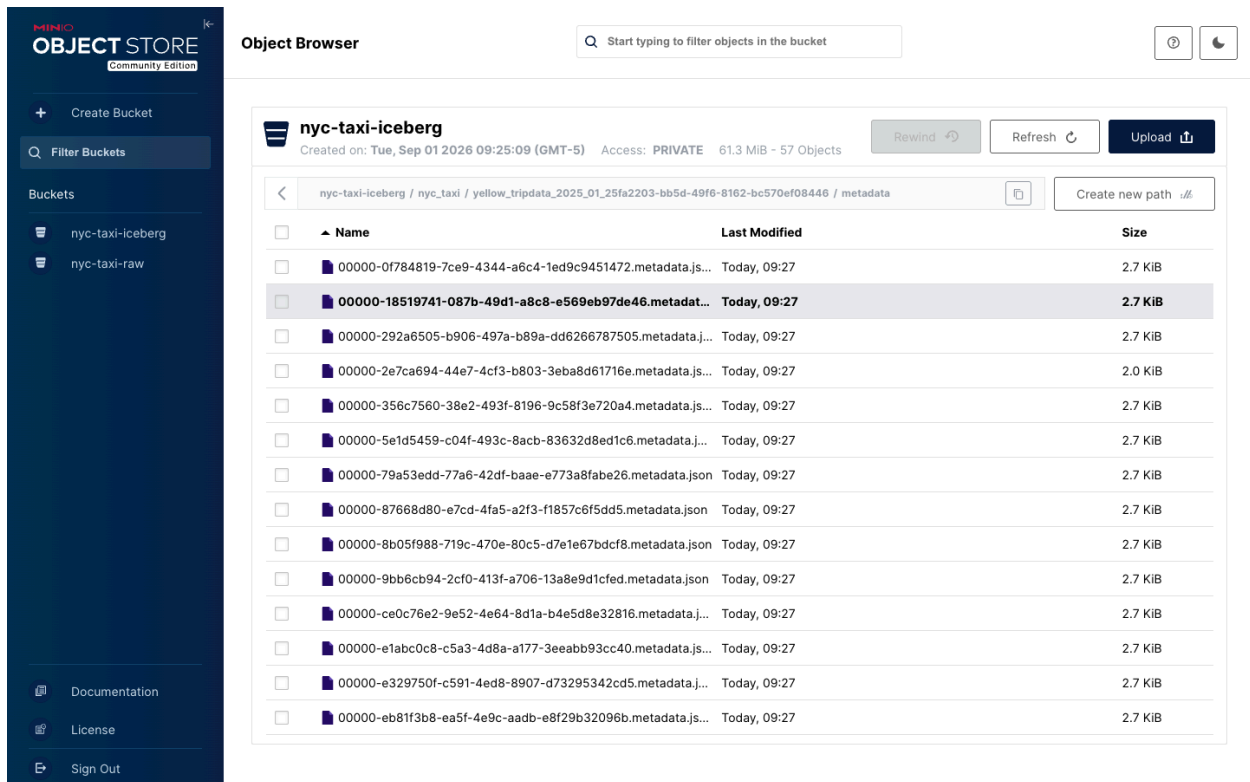


Figura 4

Metadata de la tabla Iceberg

Detalle de los archivos `.metadata.json`, `.avro` y listas de manifiestos que Iceberg genera. Esta metadata es lo que convierte un conjunto de Parquet sueltos en una tabla con snapshots e historial. Complemento del pantallazo de buckets · [evidencias/03b_minio_iceberg_metadata.png](#)

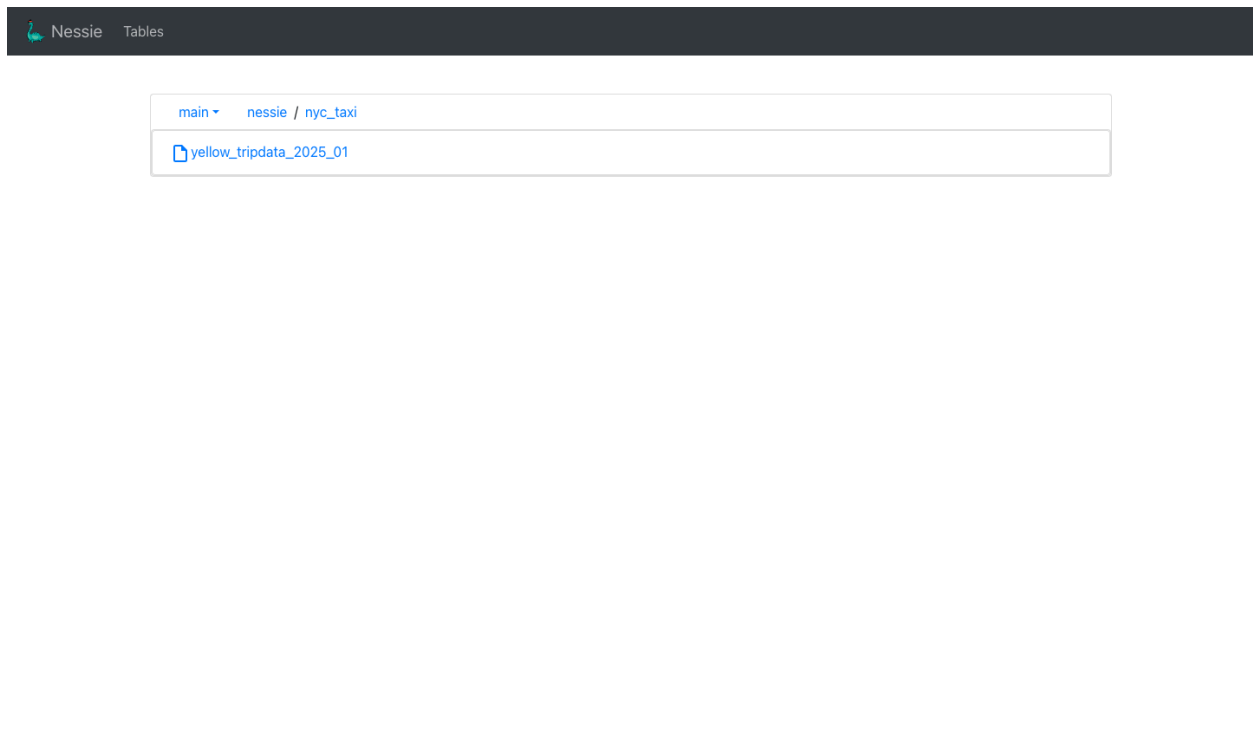


Figura 5

Catálogo Nessie con el namespace

Namespace `nyc_taxi` y tabla `yellow_tripdata_2025_01` registrados en Nessie, con su *Metadata Location* apuntando al bucket Iceberg en MinIO. Pantallazo requerido: catálogo de Nessie con el namespace creado · `evidencias/04_nessie_namespace.png`

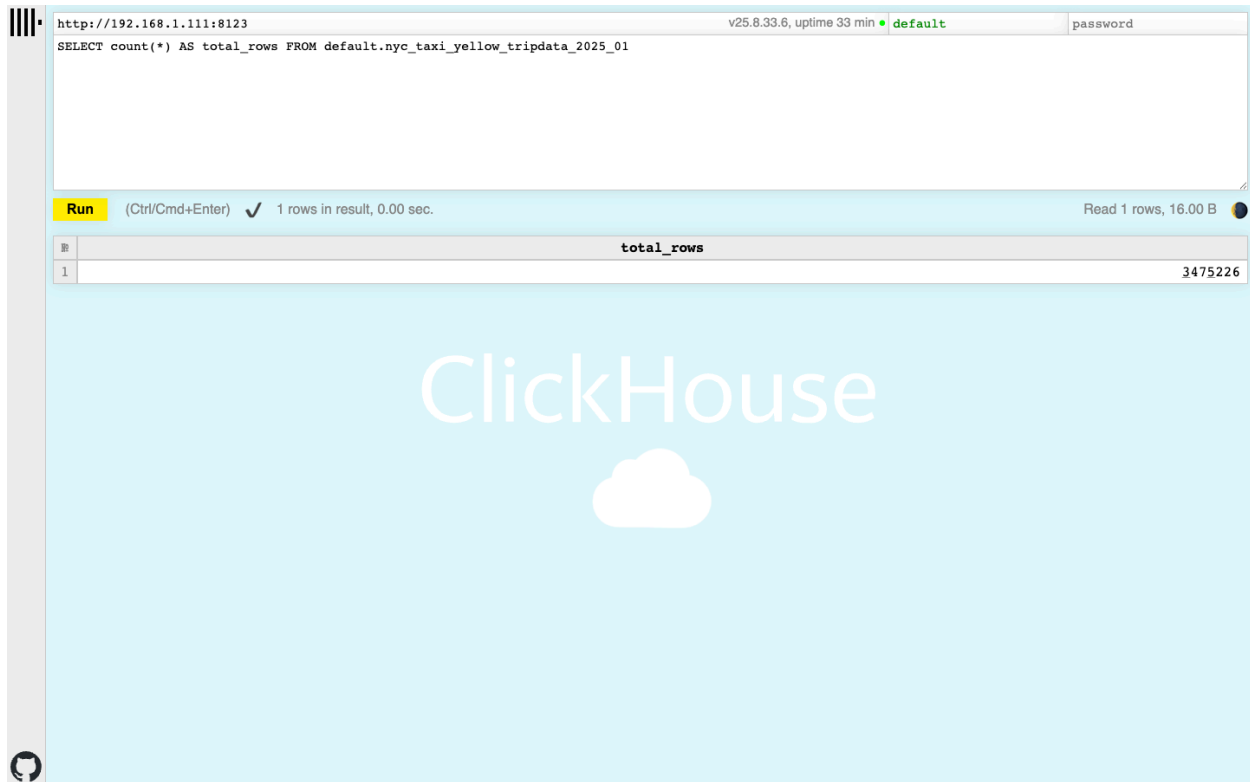


Figura 6

Conteo de registros en ClickHouse

Resultado de `SELECT count(*)` sobre la tabla final: **3.475.226 registros**, exactamente el valor exigido por el enunciado. Pantallazo requerido: consulta con cantidad de registros (3475226) · [evidencias/05_clickhouse_count.png](#)

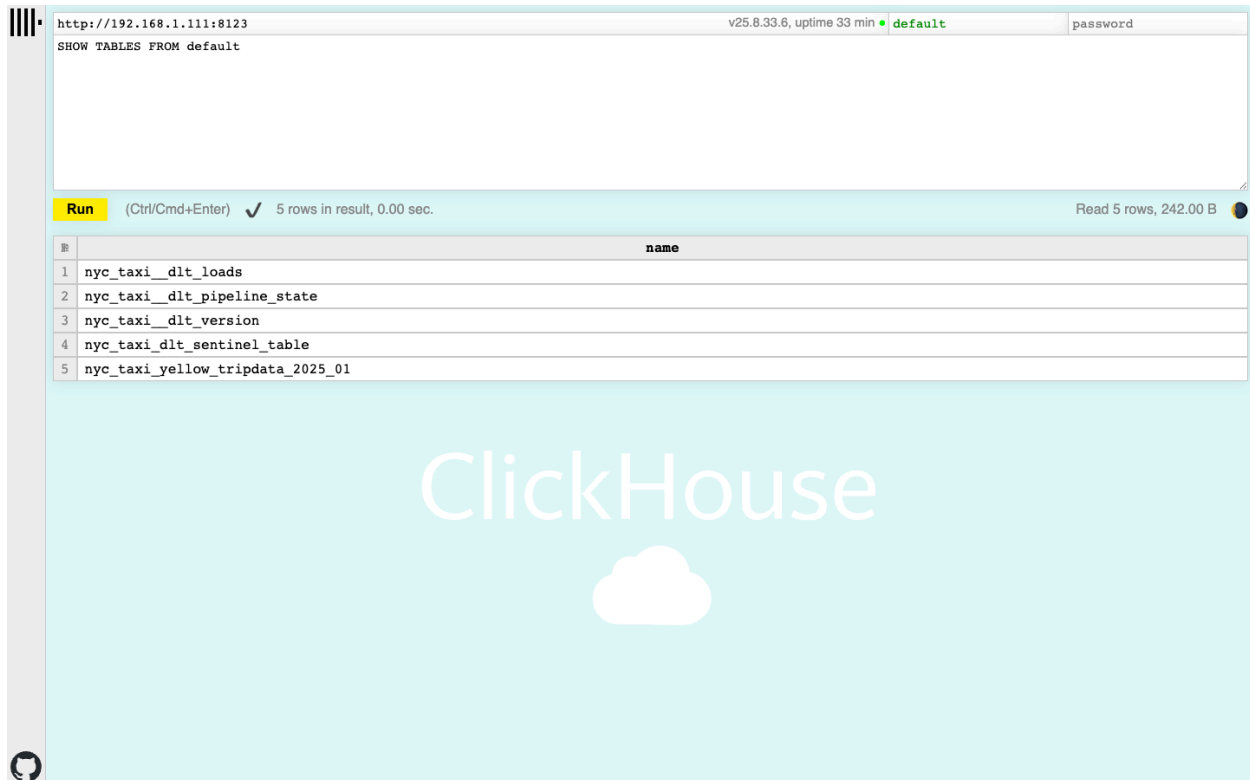


Figura 7

Salida de `SHOW TABLES FROM default` : la tabla de datos junto a las tablas de metadatos que genera `dlt` (`_dlt_loads` , `_dlt_pipeline_state` , `_dlt_version` y la tabla centinela).

Pantallazo requerido: consulta con las tablas de metadatos en ClickHouse ·

[evidencias/06_clickhouse_metadata.png](#)

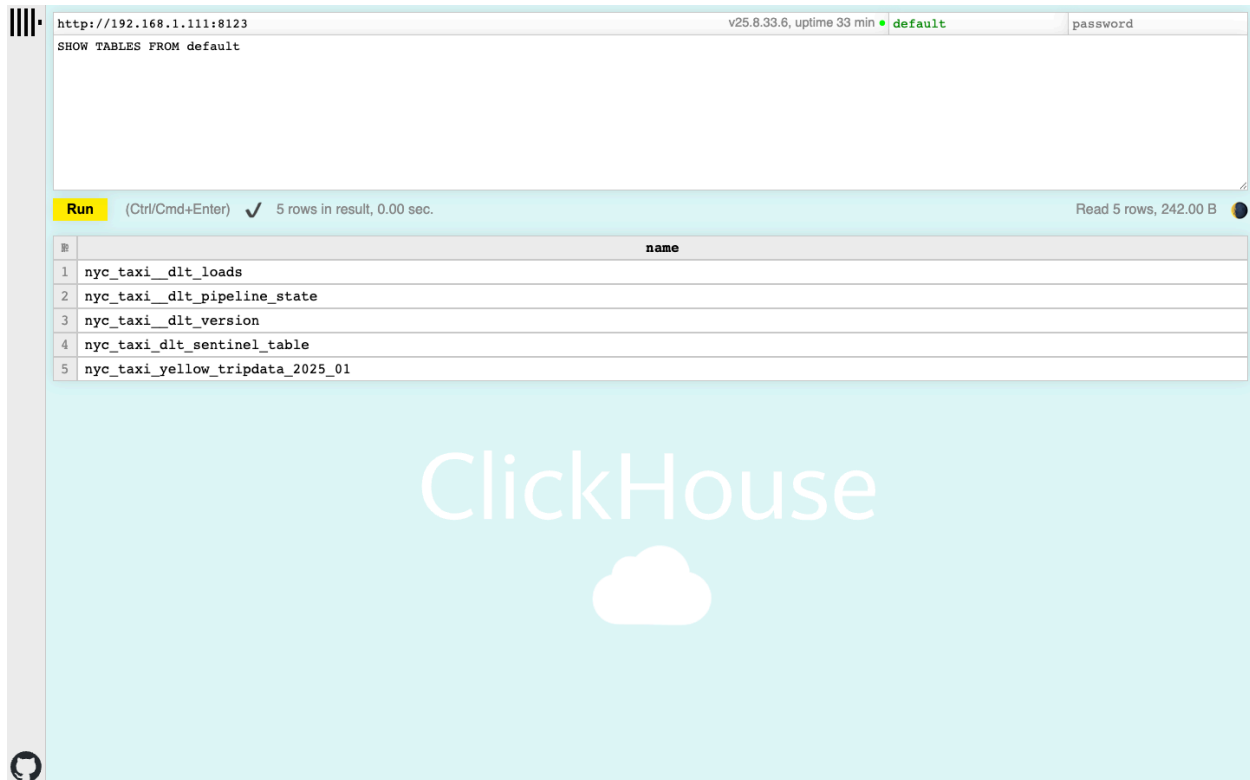


Figura 7

Tablas de metadatos en ClickHouse

Salida de `SHOW TABLES FROM default`: la tabla de datos junto a las tablas de metadatos que genera `dlt` (`_dlt_loads`, `_dlt_pipeline_state`, `_dlt_version` y la tabla centinela). Pantallazo requerido: consulta con las tablas de metadatos en ClickHouse · [evidencias/06_clickhouse_metadata.png](#)

Figura 8

Archivo cargado en Azure ADLS

Listado del contenedor `clase-4-dlt` mediante la librería `adlfs`, mostrando la carpeta `GRUPO_2/nyc_taxi/` con los data files y la metadata. La verificación es programática, sin depender del portal de Azure.

Pantallazo requerido: consulta de archivo cargado en Azure · [evidencias/07_azure_file.png](#)

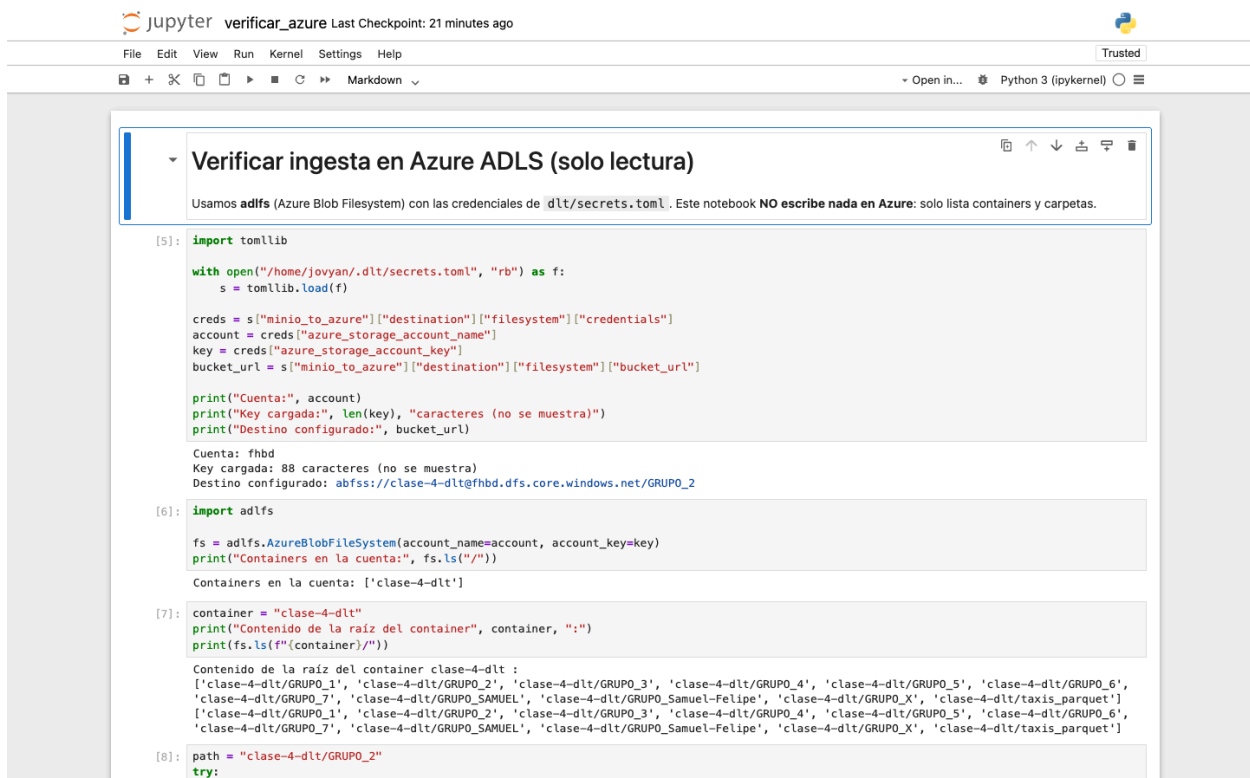


Figura 8

Archivo cargado en Azure ADLS

Listado del contenedor `clase-4-dlt` mediante la librería `adlfs`, mostrando la carpeta `GRUPO_2/nyc_taxi/` con los data files y la metadata. La verificación es programática, sin depender del portal de Azure. Pantallazo requerido: consulta de archivo cargado en Azure · `evidencias/07_azure_file.png`

Figura 9

Verificación integral: 12/12 checks

Ejecución de `verificar.py` recorriendo todo el stack en una sola corrida: buckets, archivos RAW, tabla Iceberg, catálogo Nessie, ClickHouse y Azure.

Evidencia complementaria de cierre · `evidencias/08_verificar_12de12.png`

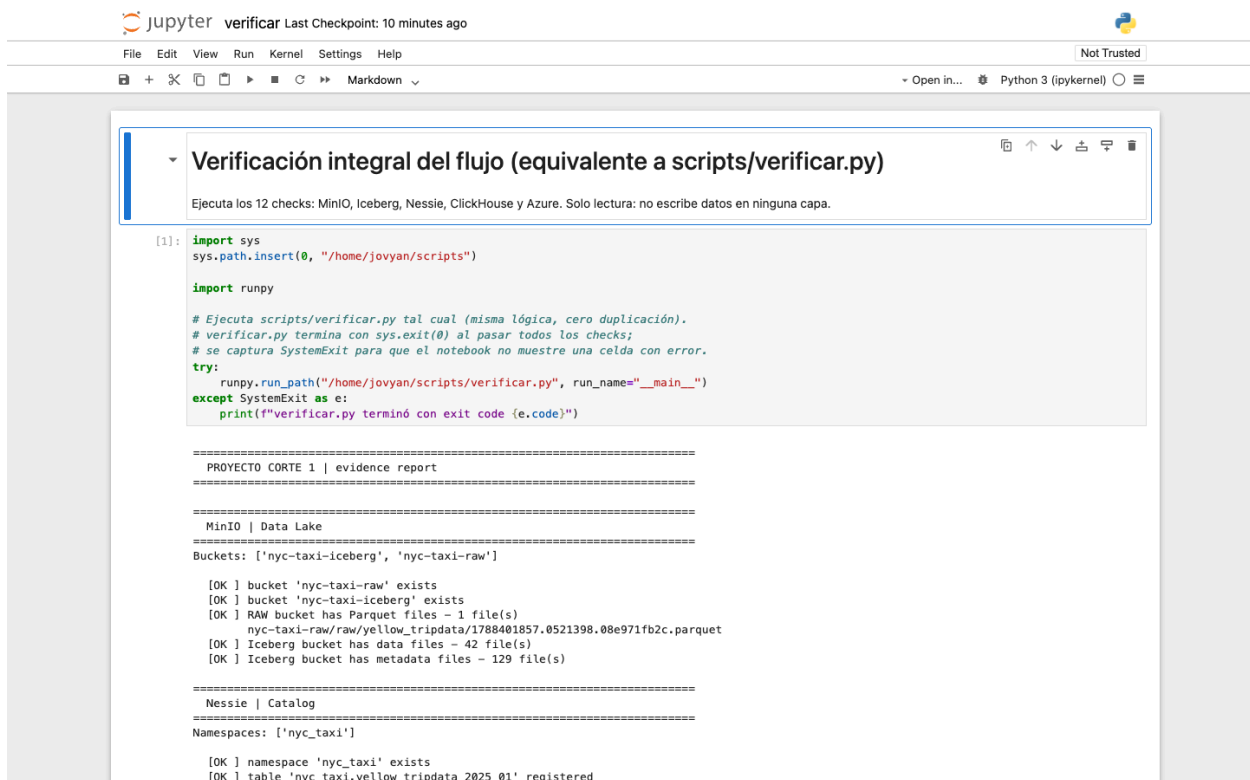


Figura 9

Verificación integral: 12/12 checks

Ejecución de `verificar.py` recorriendo todo el stack en una sola corrida: buckets, archivos RAW, tabla Iceberg, catálogo Nessie, ClickHouse y Azure. Evidencia complementaria de cierre · `evidencias/08_verificar_12de12.png`

8 Incidentes y soluciones

Incidente	Causa	Solución
Nessie fallaba con <code>Permission denied</code> en <code>/nessie/data/LOG</code>	Docker crea el volumen nombrado como <code>root:root</code> cuando la ruta no existe en la imagen; Nessie corre como usuario <code>nessie</code> (uid 10000) y no puede escribir	Servicio <code>volume-init</code> en el <code>docker-compose.yml</code> : cede la propiedad de los volúmenes antes de que arranquen los servicios que dependen de ellos
<code>PermissionError</code> en <code>/home/jovyan/state/pipelines</code>	Mismo caso en el volumen <code>jupyter-state</code> ; el contenedor corre	Cubierto por el mismo servicio <code>volume-init</code>

Incidente	Causa	Solución
	como <code>jovyan</code> (uid 1000)	
dlt rechazaba <code>secrets.toml</code> con <code>Empty key at line 1 col 0</code>	El archivo se generó en UTF-8 con BOM desde PowerShell 5.1; el parser TOML no acepta BOM	Regenerar el archivo en UTF-8 sin BOM
<code>docker compose up</code> fallaba con <code>logon session does not exist</code>	El CLI de Docker en sesión SSH no accede al credential store de la sesión interactiva de Windows	Ejecutar el <code>up</code> desde la sesión interactiva del usuario
Key de Azure con 115 caracteres en <code>secrets.toml</code>	Se concatenó texto sobrante a la key; una key válida de Azure mide 88 caracteres	Regenerar <code>secrets.toml</code> desde la plantilla con la key del enunciado, verificando longitud y hash
Azure respondía <code>AccountIsDisabled</code>	La cuenta de almacenamiento <code>fhbd</code> estaba deshabilitada del lado del servicio	El profesor re-habilitó la cuenta; el pipeline 03 completó la carga

9Reproducción desde cero

```
# dentro de la carpeta Proyecto-Corte-1-GRUPO_2
# (incluye dlt/secrets.toml y .env ya configurados)

docker compose up -d --build      # la primera build tarda unos minutos
docker ps                        # 4 contenedores Up
                                # minio-init termina en Exited 0: solo crea los buckets

# Ejecutar en orden los notebooks 01 - 02 - 03 - 04 en Jupyter
# http://localhost:8888 (token: cortel)

# o los scripts equivalentes:
docker compose exec jupyter python scripts/01_http_to_minio.py
docker compose exec jupyter python scripts/02_parquet_to_iceberg.py
```

```
docker compose exec jupyter python scripts/03_minio_to_azure.py
docker compose exec jupyter python scripts/04_minio_to_clickhouse.py
docker compose exec jupyter python scripts/verificar.py # 12/12 checks
```

Consola	URL	Credenciales
MinIO	<code>http://localhost:9001</code>	<code>admin</code> / <code>password123</code>
Nessie	<code>http://localhost:19120/tree/main/nyc_taxi</code>	—
ClickHouse (Play)	<code>http://localhost:8123/play</code>	<code>default</code> / <code>clickhouse123</code>
Jupyter	<code>http://localhost:8888</code>	token <code>cortel</code>

Los pipelines son idempotentes: re-ejecutarlos no duplica datos.

10 Conclusiones

El flujo queda completo de punta a punta: el dato público se ingiere con `dlt`, se persiste como Parquet en MinIO (data lake), se promueve a tabla Apache Iceberg con catálogo Nessie —separando file format, table format y catálogo como componentes independientes— y se distribuye a los dos destinos finales: Azure ADLS (carpeta `GRUPO_2`) y ClickHouse como data warehouse.

La validación crítica se cumple con exactitud: **3.475.226 filas** en las tres capas de datos, y la verificación integral cierra en **12/12 checks**, incluida la confirmación programática de los archivos en Azure. El stack es reproducible con un solo `docker compose up -d --build`, y los secretos permanecen fuera del código y del repositorio público en todo momento.

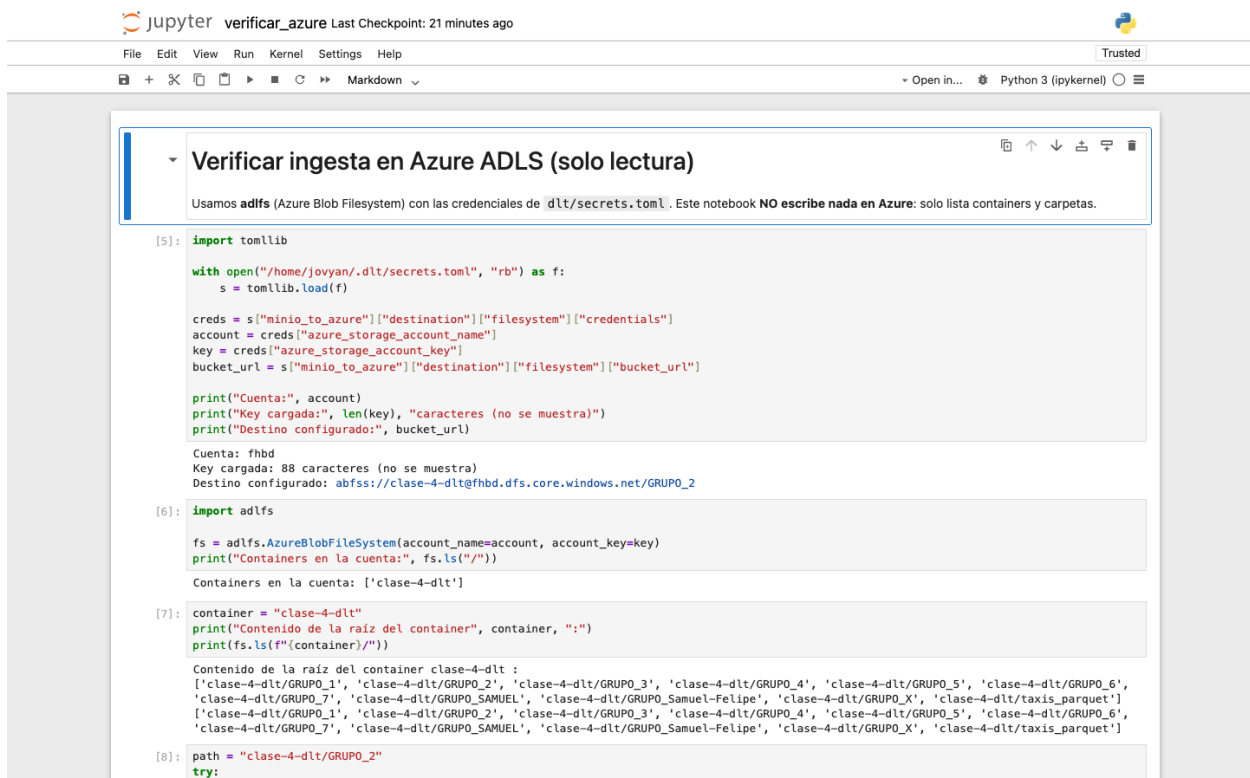


Figura 8

Listado del contenedor `clase-4-dlt` mediante la librería `adlfs`, mostrando la carpeta `GRUPO_2/nyc_taxi/` con los data files y la metadata. La verificación es programática, sin depender del portal de Azure. Pantallazo requerido: consulta de archivo cargado en Azure · `evidencias/07_azure_file.png`

Verificación integral del flujo (equivalente a scripts/verificar.py)

Ejecuta los 12 checks: MinIO, Iceberg, Nessie, ClickHouse y Azure. Solo lectura: no escribe datos en ninguna capa.

```
[1]: import sys
sys.path.insert(0, "/home/jovyan/scripts")

import runpy

# Ejecuta scripts/verificar.py tal cual (misma lógica, cero duplicación).
# verificar.py termina con sys.exit(0) al pasar todos los checks;
# se captura SystemExit para que el notebook no muestre una celda con error.
try:
    runpy.run_path("/home/jovyan/scripts/verificar.py", run_name="__main__")
except SystemExit as e:
    print(f"verificar.py terminó con exit code {e.code}")

=====
PROYECTO CORTE 1 | evidence report
=====

MinIO | Data Lake
=====
Buckets: ['nyc-taxi-iceberg', 'nyc-taxi-raw']

[OK ] bucket 'nyc-taxi-raw' exists
[OK ] bucket 'nyc-taxi-iceberg' exists
[OK ] RAW bucket has Parquet files - 1 file(s)
      nyc-taxi-raw/raw/yellow_tripdata/1788401857.0521398.08e971fb2c.parquet
[OK ] Iceberg bucket has data files - 42 file(s)
[OK ] Iceberg bucket has metadata files - 129 file(s)

=====
Nessie | Catalog
=====
Namespaces: ['nyc_taxi']

[OK ] namespace 'nyc_taxi' exists
[OK ] table 'nyc_taxi.yellow_tripdata_2025_01' registered
```

Figura 9

Ejecución de `verificar.py` recorriendo todo el stack en una sola corrida: buckets, archivos RAW, tabla Iceberg, catálogo Nessie, ClickHouse y Azure. Evidencia complementaria de cierre · [evidencias/08_verificar_12del12.png](#)

8. Incidentes y soluciones

Incidente	Causa	Solución
Nessie salía con <code>Permission denied</code> en <code>/nessie/data/LOG</code>	El volumen nombrado quedó como <code>root</code> y la imagen corre como usuario <code>nessie</code> (uid 10000).	<code>chown -R 10000:10001</code> sobre el volumen, ejecutado con <code>--user 0</code>
<code>PermissionError</code> en <code>/home/jovyan/state/pipelines</code>	Volumen <code>jupyter-state</code> con propietario <code>root</code> ; el contenedor corre como <code>jovyan</code> (uid 1000)	<code>chown -R 10000:10001</code> sobre el volumen.

Incidente	Causa	Solución
dlt rechazaba <code>secrets.toml</code> (<code>Empty key at line 1 col 0</code>)	El archivo se generó en UTF-8 con BOM desde PowerShell 5.1; el parser TOML no acepta BOM	Regenerar el archivo en UTF-8 sin BOM
<code>docker compose up</code> fallaba con <code>logon session does not exist</code>	El CLI de Docker en sesión SSH no accede al credential store de la sesión interactiva de Windows	Ejecutar el <code>up</code> desde la sesión interactiva del usuario
Key de Azure con 115 caracteres en <code>secrets.toml</code>	Se concatenó texto sobrante a la key; una key válida de Azure mide 88 caracteres	Regenerar <code>secrets.toml</code> desde la plantilla con la key del enunciado, verificando longitud y hash

9. Reproducción desde cero

```
# dentro de la carpeta Proyecto-Corte-1-GRUPO_2
# (incluye dlt/secrets.toml y .env ya configurados)
docker compose up -d --build # la primera build tarda unos m
inutos
docker ps                    # 4 contenedores Up
                             # minio-init termina en Exited
0: solo crea los buckets

# Ejecutar en orden los notebooks 01 - 02 - 03 - 04 en Jupyter
r
# http://localhost:8888 (token: corte1)
# o los scripts equivalentes:

docker compose exec jupyter python scripts/01_http_to_minio.p
y
docker compose exec jupyter python scripts/02_parquet_to_iceb
erg.py
docker compose exec jupyter python scripts/03_minio_to_azure.
```

```

py
docker compose exec jupyter python scripts/04_minio_to_clickh
ouse.py
docker compose exec jupyter python scripts/verificar.py # 12/
12 checks

```

Consola	URL	Credenciales
MinIO	http://localhost:9001	admin / password123
Nessie	http://localhost:19120/tree/main/nyc_taxi	-
ClickHouse (Play)	http://localhost:8123/play	default / clickhouse123
Jupyter	http://localhost:8888	token cortel

Los pipelines son idempotentes: re-ejecutarlos no duplica datos.

10. Conclusiones

El flujo queda completo de punta a punta: el dato público se ingiere con `dlt`, se persiste como Parquet en MinIO (data lake), se promueve a tabla Apache Iceberg con catálogo Nessie —separando file format, table format y catálogo como componentes independientes— y se distribuye a los dos destinos finales: Azure ADLS (carpeta `GRUPO_2`) y ClickHouse (data warehouse).

La validación crítica se cumple con exactitud: **3.475.226 filas** en las tres capas de datos, y la verificación integral cierra en **12/12 checks**, incluida la confirmación programática de los archivos en Azure. El stack es reproducible con un solo `docker compose up -d --build` y los secretos quedan fuera del código y del repositorio público en todo momento.